Architektury

Przetwarzanie wsadowe vs strumieniowe

- wsadowe przetwarzają w jednym przebiegu dużą ilość historycznych danych
- strumieniowe przetwarzają po jednym rekordzie na raz

	Wsadowe	**Strumieniowe**
historia	długa	wolna
prędkość	wolna	szybka
częstotliwość	niska	duża

Wyzwania w zarządzaniu architekturami Big Data

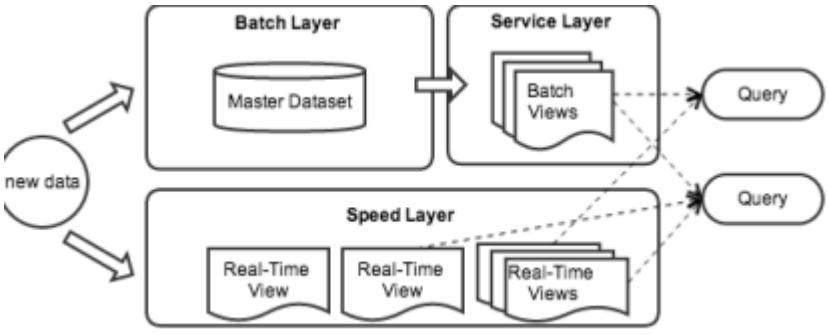
- podział komponentów architektury pomiędzy specjalizowane zespoły powoduje powstawanie silosów
- zmiany są kosztowne i rozległe
- wiele interakcji i interfejsów
- prawo Conwaya - projekt systemu IT odzwierciedla strukturę organizacyjną firmy

Architektura Data Lake



- cechy:
  - efektywna kosztowo
  - elastyczność formatów
  - opóźnienia
  - heterogeniczność źródeł danych

Architektura Lambda



- warstwy:
  - *batch-layer* - przetwarzanie danych historycznych
  - *speed-layer* - przetwarzanie w czasie rzeczywistym
  - *serving-layer* - udostępnianie widoków z przetworzonymi danymi

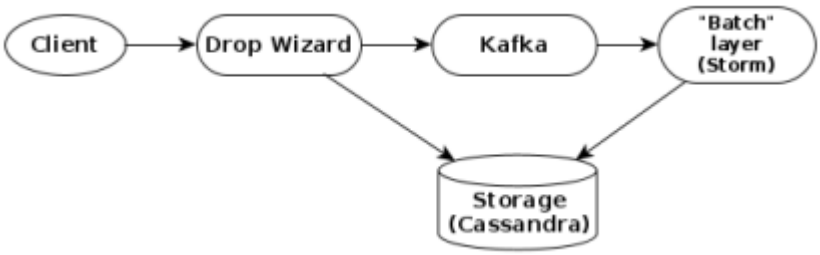
- w każdej kolejnej iteracji dane przetworzone tylko w strumieniu przetwarzane są wsadowo (w strumieniu są tylko najnowsze dane)
- zalety:
  - łączenie widokow batch i real-time
  - odporna na awarie i błędy
  - inkrementalna obsługa przyrostów danych
  - możliwość wprowadzenia usprawnień kodu które zostaną uwzględnione w kolejnym przeliczeniu dla całych danych historycznych
- wady:
  - konieczność utrzymywania dwóch strumieni (droga infrastruktura)
  - potencjalna duplikacja kodu

Przykład zastosowania

- system rekomendacji:
  - warstwa batch analizuje historię zachowań użytkowników
  - warstwa speed reaguje na bieżącą sesję
  - wynik końcowy łączy oba widoki

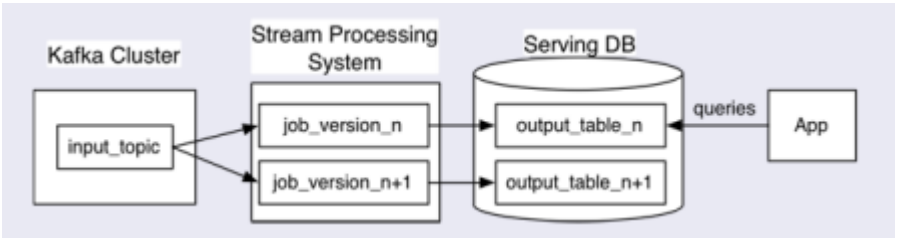
Architektura Delta

- odwrócona Lambda



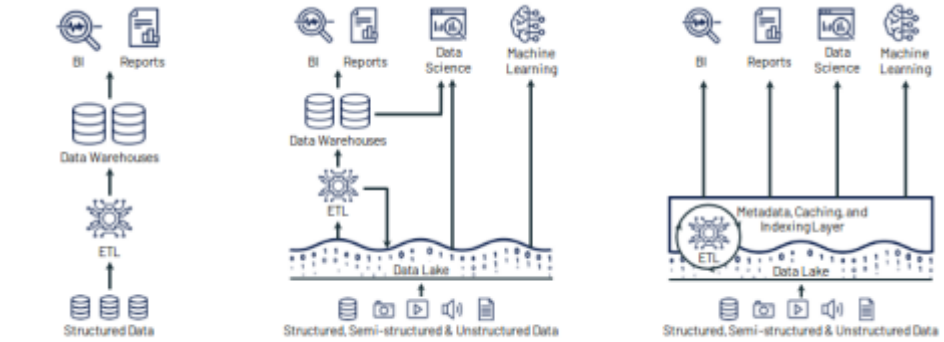
- odwraca role komponentów
- koncentruje się na przyrostach (deltach) danych

Architektura Kappa



- jedna ścieżka przetwarzania danych
- konieczność ponownego przepuszczenia danych historycznych przez algorytmy real-time
- zalety:
  - wprowadzenie nowego kodu do równoległego przetwarzania
  - możliwość przeliczenia całej dostępnej historii w modelu streaming
  - przesunięcie offsetu na początek dostępnego strumienia danych
  - przełączanie usług na nowe widoki po nadgonieniu przetwarzania przez nową wersję

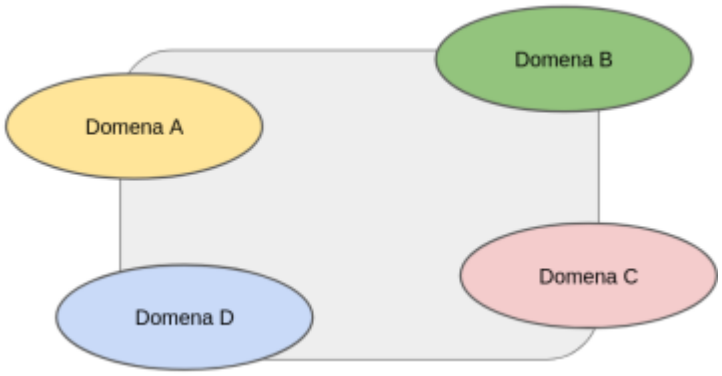
Architektura Lakehouse



(a) First-generation platforms. (b) Current two-tier architectures. (c) Lakehouse platforms.

- rozwinięcie Data Lake
- dodaje spójność, schemat, elementy transakcyjności
- dodatkowy narzut na scentralizowane zarządzanie danymi
- tworzy silosy danych

Data Mesh



- zdecentralizowana odpowiedzialność za dane z różnych domen
- dane jako produkt dostarczany przez różne zespoły
- skalowalność, niezależność, *Domain-Driven Design*
- przykład - e-commerce:
  - domeny:
    - klient (konta, autentykacja itp.)
    - zamówienia (koszyk, płatność itp.)
    - produkt (katalog, stany magazynowe itp.)
    - analityka (segmentacja klientów, rekomendacje itp.)
  - każdy zespół odpowiedzialny za:
    - jakość danych
    - schemat
    - dokumentację
    - bezpieczeństwo
    - transformacje

Spójne przetwarzanie zdarzeń

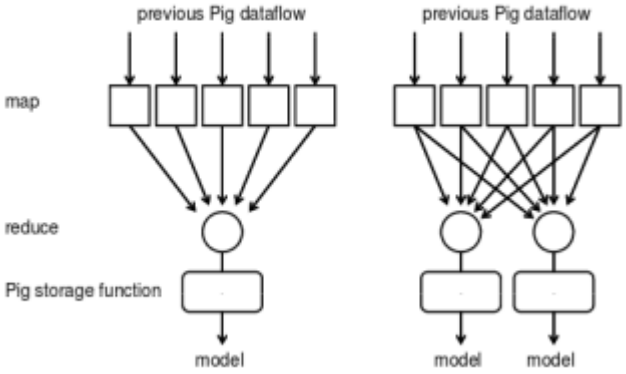
- celem może być uzyskanie semantyki *exactly-once*
- często wyniki publikowane są w stałych interwałach, np. co godzinę
- ważne pojęcia:

- *offset* - pozycja w strumieniu
- czas zdarzenia i czas przetwarzania

SQL na danych rozproszonych

- Hive / Tez
  - wykonywanie zapytań SQL w środowisku Hadoop
- SparkSQL
  - SQL nad danymi przetwarzanymi wsadowo i strumieniowo

Data mining w Twitter



- zastosowanie języka przepływu danych wysokiego poziomu Pig
- podział transformacji na re-używalne biblioteki
- wykorzystanie orkiestracji danych

Inne przykłady architektur analitycznych

Starfish

- optymalizacje dla środowiska Hadoop
- strojenie parametrów zadań i planu wykonania
- optymalizacja serializacji i współdzielenia danych

Kylin

- interfejs SQL i analityka OLAP nad Hadoop
- prekalkulacja agregatów dla szybkich zapytań analitycznych

MLBase

- przykład architektury do zastosowań uczenia maszynowego

Bazy danych NoSQL

Bazy key-value

- przechowują pary klucz-wartość
- nie wymagają predefiniowanej struktury
- optymalizacja na szybkość odpytywania pojedynczych rekordów po kluczu
- np. obsługa sesji

Bazy kolumnowe

- dane grupowane po kolumnach a nie wierszach:
  - baza wierszowa na dysku: `[1, Anna, 30, Warszawa] [2, Piotr, 25, Kraków]`
  - baza kolumnowa na dysku: `[1, 2] [Anna, Piotr] [30, 25] [Warszawa, Kraków]`

- luźny schemat (np. dodawanie nowych kolumn w locie)
- zoptymalizowane na przetwarzanie całych kolumn (np. w zapytaniach agregujących `sum`, `max`, `avg`)

Bazy dokumentowe
- pary klucz-wartość w formie dokumentów
- dokumenty grupowane w kolekcje
- duża elastyczność bez ścisłego formatu

Bazy grafowe
- dane w postaci wierzchołków i krawędzi
- dobrze odzwierciedlają relacje
- zoptymalizowane po przeszukiwanie grafu

(De)serializacja
- **serializacja** - proces konwersji struktur danych na strumień bitów w celu zapisu na dysk, przesłania do innych węzłów itp.
- **deserializacja** - proces odwrotny

Formaty tesktowe
- np. XML, CSV, JSON, YAML
- kodują każdą wartość danych jako znaki ASCII/UTF-8
  - np. liczba `30` to są 2 bajty `0x33` i `0x30`
- czytelne dla człowieka
- opcjonalny schemat
- wolne parsowanie

Formaty binarne
- np. protobuf, MessagePack
- zapis typów i wartości bezpośrednio jako bajty
  - np. liczba `30` jest zapisana bezpośrednio jako `0x1E`
- ekstremalnie szybkie parsowanie

Porównania praktyczne
- formaty tekstowe vs binarne:
  - tekstowe są bardziej czytelne dla człowieka
  - binarne są zwykle szybsze i bardziej kompaktowe
- popularne formaty w ekosystemie Hadoop:
  - Avro - format wierszowy, dobry do wymiany danych
  - Parquet - format kolumnowy, dobry do analityki

Orkiestracja danych
- zbiór technologii które wprowadzają abstrakcję danych ponad warstwę ich przechowywania
- zalety:
  - ograniczenie powtarzalnej pracy
  - skracanie czasu na tworzenie nowych usług
  - ułatwienie zmiany technologii
  - płynna migracja do chmury